Day 119: k-Means From Scratch, and the Elbow Method Tested

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 1 | Complete study guide

Watch: StatQuest: K-means clustering (StatQuest with Josh Starmer) | K-means clustering: how it works (Victor Lavrenko)

Script: `day119_k_means.py` · every number in this guide is copied by program from `day119_run_output.txt`, the real output of that script.

Week 18 begins unsupervised learning: finding groups without labels. k-means is written from scratch, checked against scikit-learn, and its best-known heuristic, the elbow, is tested on three real datasets whose true groups are known. The elbow finds k = 3 on iris and wine, but on digits, with 10 true groups, it suggests k = 5. Scaling decides the result on wine (0.3711 raw, 0.8975 standardised). Without a single label, k-means sorts 0.7896 of handwritten digits into the right group.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

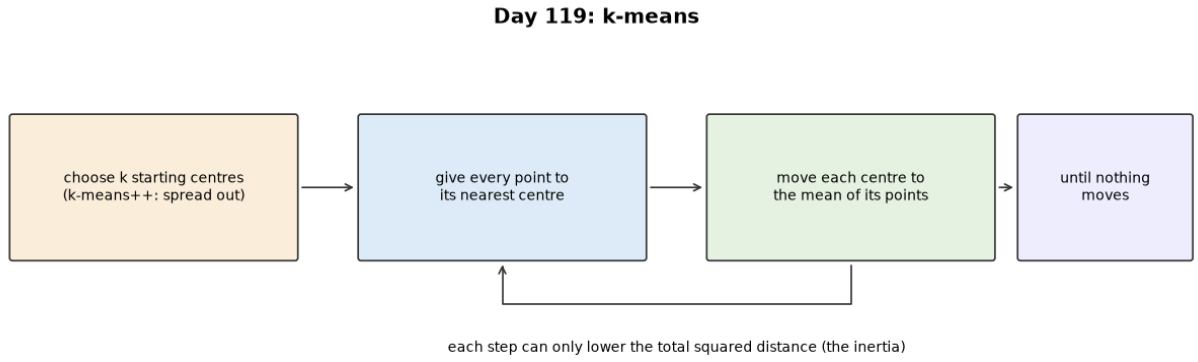
Clustering finds groups in data with no labels. **k-means** keeps k **centres**; it assigns every point to its nearest centre, moves each centre to the mean of its points, and repeats until nothing changes. It minimises the **inertia**: the total squared distance from points to their centres.

Because the true groups are known for today's datasets, the result can be scored with the **adjusted Rand index** (ARI): 1 for perfect agreement, about 0 for random labels. The **silhouette** scores a clustering without knowing the truth.

REAL-WORLD SCENARIO

A shop with a million customers and no categories: k-means groups them by what they buy, and the marketing team then names the groups. Nobody told the algorithm what a 'bargain hunter' is.

SYSTEM DIAGRAM



The loop: assign each point to its nearest centre, move each centre to its points' mean, repeat.

Today's part for the week's build

`day119_exercise.py` is the loop, the ARI, the silhouette and the elbow, checked by hand. Its real output:

ACTUAL OUTPUT

```
six points from a poor start: groups [[1.0, 2.0, 3.0], [10.0, 11.0, 12.0]], centres [2.0, 11.0],
    inertia by step [246.0, 41.68, 4.0]
by hand: centres 2 and 11, inertia (1 + 0 + 1) + (1 + 0 + 1) = 4
ARI ignores the numbering: [0,0,1,1] against [1,1,0,0] -> 1.000
a random labelling against the truth, averaged over 20 tries: -0.0001
silhouette of the six points in their two groups: 0.8505 (tight groups far apart: near 1)
elbow of a curve that falls fast until k = 4 and slowly after: k = 4
```

Data: iris (150 flowers, 3 species) and wine (178 wines, 3 cultivars), both standardised, and digits (1,797 images, 10 digits) as raw pixel values from 0 to 16, all bundled with scikit-learn. scikit-learn's k-means and scores are an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: six points, two centres

DEFINITION — start here, no prior knowledge assumed

Points 1, 2, 3, 10, 11, 12 on a line, and a poor start: both centres on the left, at 1 and 2.

REAL-WORLD SCENARIO

Two ice-cream vans parked at the same end of a beach: each step, each van moves to the middle of its own customers.

MATHEMATICS

$$inertia = \Sigma_i \, |x_i - c_{label(i)}|^2; \quad c_j = mean\ of\ the\ points\ labelled\ j$$

IMPLEMENTATION

```
def lloyd(X, C, max_iter=300):
    history = []
    for _ in range(max_iter):
        D = sqdist(X, C); lab = D.argmin(1); history.append(float(np.maximum(D.min(1), 0).sum()))
        new = np.array([X[lab == j].mean(0) if np.any(lab == j) else C[j] for j in range(len(C))])
        if np.allclose(new, C):
            break
        C = new
    return lab, C, history
```

ACTUAL OUTPUT

```
points 1, 2, 3, 10, 11, 12 on a line; centres start at 1 and 2
  step 1: groups [[1.0], [2.0, 3.0, 10.0, 11.0, 12.0]], inertia 246.00; new centres [1.0, 7.6]
  step 2: groups [[1.0, 2.0, 3.0], [10.0, 11.0, 12.0]], inertia 41.68; new centres [2.0, 11.0]
  step 3: groups [[1.0, 2.0, 3.0], [10.0, 11.0, 12.0]], inertia 4.00; new centres [2.0, 11.0]
by hand: the best split is {1, 2, 3} and {10, 11, 12}, centres 2 and 11, inertia 2 + 2 = 4
Actual output from this lesson's run.
```

What it means

Step 1 puts only the point 1 in the left group (inertia 246.00), and the right centre moves to 7.6. Step 2 finds the natural split, and step 3 confirms it: centres 2 and 11, inertia 4.00, exactly the hand answer. Each step can only lower the inertia: reassigning a point to a nearer centre lowers it, and the mean is the point with the least squared distance to a group.

WHY CHOOSE THIS?

A loop simple enough to run by hand.

WHY NOT THIS? (TRADEOFFS)

A poor start can still end somewhere worse than this; section 3 measures how often.

2. Correctness

DEFINITION — start here, no prior knowledge assumed

Three checks: the inertia never rises over 20 runs; from the same starting centres the result matches scikit-learn's; and the ARI and silhouette match scikit-learn's to six decimals.

REAL-WORLD SCENARIO

Checking a home-made scale against the shop's, with the same weights.

ACTUAL OUTPUT

```
digits, k = 10, 20 random starts: 381 steps in all; steps where the inertia went up: 0
from the same starting centres: inertia 1167803.84 here, 1167791.56 in scikit-learn; adjusted Rand
    index between the two labellings 0.9972
why they differ: at step 1, 2 image(s) lie exactly as far from two centres (image 23: squared
    distance 1432 to both); pixels are whole numbers, so exact ties happen, and rounding decides
the same steps on mean-centred data, as scikit-learn computes them: inertia 1167791.56; ARI with
    scikit-learn's labelling 1.0000
silhouette 0.187974 against scikit-learn's 0.187974; ARI with the true digits 0.652481 against
    0.652481
Actual output from this lesson's run.
```

What it means

The inertia went up on 0 of 381 steps. From the same start, the two versions end close but not identical: 1167803.84 against 1167791.56, with labellings that agree at 0.9972. The script found why. At step 1, image 23 is at squared distance 1432 from *two* centres: pixels are whole numbers, so exact ties happen, and which centre wins depends on rounding. scikit-learn subtracts the mean of the data first; running the same steps on mean-centred data reproduces its result exactly (1.0000). Both answers are correct; they are different local minima reached through one tie.

WHY CHOOSE THIS?

Every claim later in the lesson rests on a checked loop and checked scores.

WHY NOT THIS? (TRADEOFFS)

Ties make two correct versions disagree; exact agreement is not the right test.

3. Starting points: random against k-means++

DEFINITION — start here, no prior knowledge assumed

k-means only finds a local minimum, so the start matters. **k-means++** picks the first centre at random and each next one far from those already chosen, with probability proportional to squared distance. 20 single starts of each, on digits.

REAL-WORLD SCENARIO

Opening ten shops: spreading them out at the start beats dropping all ten in random spots, or so the theory says.

MATHEMATICS

$$P(\text{next centre} = x) = D(x)^2 / \Sigma D^2, \quad D(x) = \text{distance from } x \text{ to the nearest centre chosen so far}$$

IMPLEMENTATION

```
def init_centres(X, k, rng, method):
    if method == "random":
        return X[rng.choice(len(X), k, replace=False)].copy()
    C = [X[rng.randint(len(X))]]
    for _ in range(k - 1):
        d = np.maximum(sqdist(X, np.array(C)).min(1), 0)
        C.append(X[rng.choice(len(X), p=d / d.sum())])
    return np.array(C)
```

ACTUAL OUTPUT

```
  random    : inertia 1165142 to 1243340 (mean 1183979); ARI with the digits 0.4682 to 0.7086;
      steps 20.1
  k-means++ : inertia 1165156 to 1261017 (mean 1193076); ARI with the digits 0.4612 to 0.7435;
      steps 20.3
  the best of 10 k-means++ starts: inertia 1167804, ARI 0.6525
every start ends in a local minimum; different starts end in different ones, which is why restarts
    are standard
Actual output from this lesson's run.
```

What it means

Here k-means++ was no better: its mean inertia was 1193076 against 1183979 for random starts, and both ranges overlap almost entirely. Its guarantee is a bound on the expected cost (within a factor of order log k of the best), not a promise to beat random rows. What clearly helps is restarting: the best of 10 k-means++ starts reaches 1167804, near the best single start of either kind (1165142). The ARI ranges show how much is at stake: one start agreed with the digits at 0.4682, another at 0.7086.

WHY CHOOSE THIS?

Restarts are cheap and remove most of the luck.

WHY NOT THIS? (TRADEOFFS)

k-means++ costs a pass over the data per centre, and did not pay for itself here.

4. The elbow, on three real datasets

DEFINITION — start here, no prior knowledge assumed

Run k-means for k = 1 to 10 and plot the inertia. It always falls as k grows; the 'elbow' is where it stops falling fast. Here the elbow is found by a rule (the point furthest below the straight line from k = 1 to k = 10), and compared with the silhouette and with the truth.

REAL-WORLD SCENARIO

Hiring more cashiers shortens the queue: the first few help a lot, then each extra one helps less. Where is the bend?

MATHEMATICS

silhouette(i) = (b – a) / max(a, b): a = mean distance to its own cluster, b = to the nearest other cluster

ARI = (pairs agreeing – expected by chance) / (maximum – expected by chance)

IMPLEMENTATION

```
def silhouette(X, lab):
    D = np.sqrt(np.maximum(sqdist(X, X), 0)); s = np.zeros(len(X)); ks = np.unique(lab)
    for i in range(len(X)):
        same = lab == lab[i]; n = same.sum() - 1
        if n == 0:
            continue
        a = D[i, same].sum() / n
        b = min(D[i, lab == c].mean() for c in ks if c != lab[i])
        s[i] = (b - a) / max(a, b)
    return float(s.mean())
def ari(a, b):
    _, ia = np.unique(a, return_inverse=True); _, ib = np.unique(b, return_inverse=True)
    T = np.zeros((ia.max() + 1, ib.max() + 1)); np.add.at(T, (ia, ib), 1)
    c2 = lambda x: x * (x - 1) / 2
    s = c2(T).sum(); sa = c2(T.sum(1)).sum(); sb = c2(T.sum(0)).sum(); e = sa * sb / c2(len(a))
    return float((s - e) / (0.5 * (sa + sb) - e))
def elbow(ks, inertia):
    x = np.array(ks, float); y = np.array(inertia, float)
    x, y = (x - x[0]) / (x[-1] - x[0]), (y - y[-1]) / (y[0] - y[-1])
    return ks[int(np.argmax((1 - x) - y))]
```

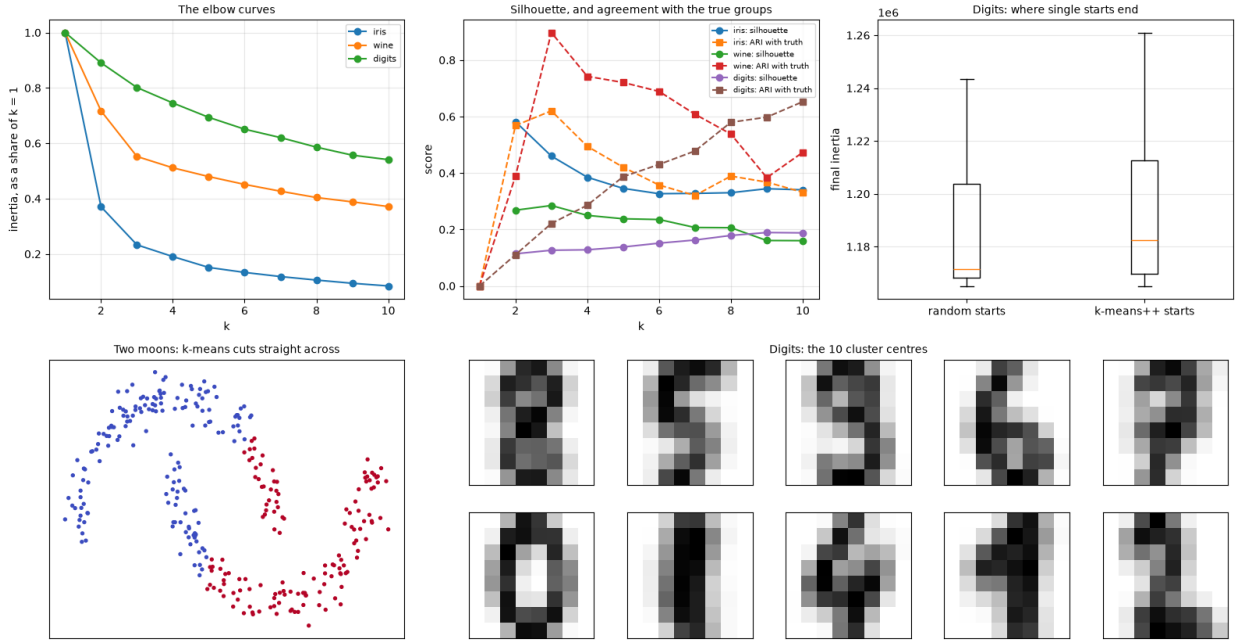
ACTUAL OUTPUT

```
k-means++ with 10 restarts for every k; the true number of groups is known (3, 3 and 10) but never shown to
    k-means
iris: 150 rows, 4 features, 3 true groups
     k     inertia     drop  silhouette     ARI
     1       600.0                  nan  0.0000
     2       222.4    0.629      0.5818  0.5681
     3       139.8    0.371      0.4599  0.6201
     4       114.4    0.182      0.3850  0.4945
     5        90.8    0.206      0.3455  0.4205
     6        80.0    0.119      0.3267  0.3572
     7        71.0    0.112      0.3277  0.3204
     8        63.4    0.107      0.3303  0.3891
     9        56.5    0.109      0.3443  0.3677
    10        50.7    0.103      0.3404  0.3318
  the automatic elbow: k = 3; best silhouette: k = 2; best agreement with the truth: k = 3
wine: 178 rows, 13 features, 3 true groups
     k     inertia     drop  silhouette     ARI
     1      2314.0                  nan  0.0000
     2      1659.0    0.283      0.2683  0.3887
     3      1277.9    0.230      0.2849  0.8975
     4      1183.1    0.074      0.2500  0.7424
     5      1109.7    0.062      0.2379  0.7205
     6      1044.3    0.059      0.2353  0.6890
     7       987.3    0.055      0.2070  0.6081
     8       934.7    0.053      0.2063  0.5380
```

```
     9       898.4    0.039      0.1609  0.3835
    10       859.5    0.043      0.1604  0.4730
  the automatic elbow: k = 3; best silhouette: k = 3; best agreement with the truth: k = 3
```

ACTUAL OUTPUT

```
digits: 1797 rows, 64 features, 10 true groups
     k     inertia     drop  silhouette     ARI
     1   2159057.3                  nan  0.0000
     2   1922853.1    0.109      0.1139  0.1106
     3   1730176.5    0.100      0.1265  0.2211
     4   1609622.2    0.070      0.1280  0.2858
     5   1497593.8    0.070      0.1379  0.3869
     6   1404985.5    0.062      0.1515  0.4303
     7   1339432.6    0.047      0.1626  0.4780
     8   1265042.2    0.056      0.1789  0.5797
     9   1202297.9    0.050      0.1892  0.5977
    10   1167803.8    0.029      0.1880  0.6525
  the automatic elbow: k = 5; best silhouette: k = 9; best agreement with the truth: k = 10
Actual output from this lesson's run.
```



Top row: the elbow curves; the three scores for each k; inertia from 20 random and 20 k-means++ starts. Bottom: k-means on two moons, and the ten digit centres.

What it means

On iris and wine the elbow finds 3, the true number. On digits it fails: the curve bends gently all the way, the rule picks k = 5, the silhouette picks k = 9, and only the truth says 10. The fall from k = 9 to 10 is the smallest in the table (0.029), yet k = 10 agrees best with the digits (0.6525). On iris the silhouette prefers 2 (0.5818), because two of the three species overlap. With real data, the 'right' k is often a choice made for a purpose, not a fact the data announces.

WHY CHOOSE THIS?

A quick picture of how much each extra cluster buys.

WHY NOT THIS? (TRADEOFFS)

On data without a sharp bend, the elbow is a matter of taste.

5. Scaling

DEFINITION — start here, no prior knowledge assumed

k-means uses distances, so a feature measured in large units dominates. On wine, 'proline' has a spread of 314.0, against 14.2 for the next largest feature.

REAL-WORLD SCENARIO

Comparing houses by price in pounds and size in rooms: without rescaling, only the price matters.

ACTUAL OUTPUT

```
'proline' has spread 314.0; the next largest, 'magnesium', 14.2; the smallest 0.124
  raw          : ARI with the 3 cultivars 0.3711
  standardised : ARI with the 3 cultivars 0.8975
Actual output from this lesson's run.
```

What it means

Raw, the distances are dominated by proline, and k-means agrees with the cultivars at 0.3711; standardised, at 0.8975. Unlike trees (Day 111), k-means needs its features on comparable scales, and choosing that scale is part of the model.

WHY CHOOSE THIS?

Standardising is one line and changed the answer completely.

WHY NOT THIS? (TRADEOFFS)

It also gives noise features the same weight as useful ones.

6. Where k-means fails

DEFINITION — start here, no prior knowledge assumed

Two shapes made on purpose, so the true groups are known exactly: two interleaved half-moons, and a wide cluster of 450 beside a tight one of 50.

REAL-WORLD SCENARIO

Drawing a straight fence to separate two curled-up cats.

ACTUAL OUTPUT

```
  two interleaved moons                  : ARI 0.2475; cluster sizes [161, 139] (true [150, 150])
  a wide cluster beside a small tight one : ARI 0.2036; cluster sizes [175, 325] (true [450, 50])
k-means draws straight boundaries halfway between centres, so it cuts curved groups and splits big
    groups to balance distances
Actual output from this lesson's run.
```

What it means

Both fail. The moons are cut straight across (0.2475), because k-means draws straight boundaries halfway between centres. The wide cluster is split to balance distances, giving sizes [175, 325] instead of 450 and 50 (0.2036). k-means assumes round groups of similar size. Day 121's DBSCAN is built for the shapes it cannot represent.

WHY CHOOSE THIS?

Shows exactly which assumption breaks.

WHY NOT THIS? (TRADEOFFS)

Synthetic data; real failures are rarely this clean to see.

7. What the clusters look like

DEFINITION — start here, no prior knowledge assumed

Digits with k = 10, each cluster labelled afterwards by its most common digit. The centres are themselves images: the average of each cluster's pixels.

REAL-WORLD SCENARIO

Sorting a pile of unlabelled post by handwriting alone, then reading one letter from each pile.

ACTUAL OUTPUT

```
 cluster  size  most common digit   share
       5   179                  0   0.989
       7   162                  4   0.988
       3   181                  6   0.972
       1   151                  5   0.927
       4   201                  7   0.861
       9   175                  2   0.829
       6   123                  1   0.813
       0   168                  8   0.798
       8    91                  1   0.593
       2   366                  3   0.437
labelling each cluster by its most common digit gives 0.7896 of images right, with no labels used
    to form the clusters
Actual output from this lesson's run.
```

What it means

Some clusters are almost pure: zeros at 0.989, fours at 0.988. Others mix: the largest cluster (366 images) is only 0.437 threes, and the ones are split across two clusters. Overall 0.7896 of images land in a cluster whose majority is their own digit, with no labels used. The centre images in the figure show why: most look like blurred but recognisable digits.

WHY CHOOSE THIS?

Unlabelled data can still be organised usefully.

WHY NOT THIS? (TRADEOFFS)

Labelling clusters afterwards needs labels; this score is a check, not a classifier.

Interview preparation — technical questions

Q: How does k-means work?

A: Assign each point to its nearest centre, move each centre to its points' mean, repeat; each step lowers the inertia (0 rises in 381 steps).

Q: How do you choose k?

A: Elbow, silhouette and the purpose; on digits the elbow said 5 and the silhouette 9, the truth was 10.

Q: Why scale features before k-means?

A: Distances are dominated by large units: wine 0.3711 raw, 0.8975 standardised.

Q: Why run k-means several times?

A: Each start ends in a local minimum; ARI ranged from 0.4682 to 0.7086 across single starts.

Q: How do you compare two clusterings?

A: The adjusted Rand index: it ignores label numbers and corrects for chance.

Interview preparation — theoretical questions

Q: Why does the inertia never rise?

A: Reassigning to a nearer centre and moving a centre to the mean can each only lower it.

Q: What does k-means++ do?

A: Spreads the starting centres with probability proportional to squared distance; here no better than random rows.

Q: What shapes can k-means find?

A: Round groups of similar size: it cut the moons straight across (ARI 0.2475).

Q: Why can two correct k-means codes disagree?

A: Exact ties, broken by rounding, lead to different local minima: one image tied at 1432.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> k-means is a short loop that always lowers its cost and always stops, but where it stops depends on the start, the scale of the features and the shape of the groups. The elbow worked on two datasets and failed on the third: the number of clusters is a decision, best made with several scores and a purpose in mind.

- Elbow: k = 3 on iris and wine; k = 5 on digits (true 10).
- Wine: ARI 0.3711 raw, 0.8975 standardised.
- Digits: 0.7896 of images in a cluster led by their own digit, no labels used.
- scikit-learn reproduced exactly once an exact tie is handled the same way.

Suggested watch order, if starting from zero

- 1. StatQuest: K-means clustering (StatQuest with Josh Starmer) — k-means step by step, and choosing k.
- 2. K-means clustering: how it works (Victor Lavrenko) — a lecture on the algorithm and its local minima.

Day 120 preview

- **Hierarchical clustering from scratch**: no k chosen in advance, but a whole tree of merges, drawn as a real dendrogram.